PRD — Graph Giraffe: Visual Backend Design & Code Generation Tool

Status: Draft **Owner:** Daniel Emmanuel **Related:** RFC-0001 — Graph Giraffe Platform Architecture **Target Release:** v1.0 (Backend DSL scope only — see §11 Out of Scope)

---

1. Summary

Graph Giraffe is a visual, node-based editor for designing backend systems — API routes, data models, business logic, integrations — either by hand or via AI-assisted graph generation. The graph is the source of truth; a deterministic compiler transforms it into production-grade backend code (initially TypeScript/Node).

This PRD scopes the **first shippable product**: a node editor + Backend DSL + compiler pipeline that takes a user's visual design (manual or AI-generated) and outputs a working, deployable backend codebase.

It builds directly on RFC-0001's layered architecture but narrows scope to a single semantic domain (Backend), deferring AI workflows, rendering pipelines, ETL, and other DSLs to later phases.

---

2. Problem Statement

Building a backend today means translating a mental model (routes, entities, flows, side effects) directly into code, with the design living only in the developer's head or in documentation that drifts from reality. Low-code tools solve part of this but usually:

- Generate opaque, unmaintainable output, or
- Trap the user inside a proprietary runtime, or
- Conflate the editor, the domain, and the execution engine so tightly that the tool can't grow past its original use case.

Teams want to **design** a backend visually — see the routes, the data flow, the validation, the side effects — and get back **real, readable, deployable code** they own, not a black box.

---

3. Goals & Non-Goals

Goals

- Let a user visually compose a backend (routes, persistence, flow, effects, error handling) as a graph.
- Let a user generate or complete that graph via an AI prompt, then edit it manually.
- Compile the graph deterministically into working backend source code.
- Validate the design at compile time (types, contracts, missing dependencies, cycles) before code is generated.
- Ship generated code that is idiomatic, readable, and usable outside the tool.
- Keep the editor itself free of backend-specific knowledge — domain logic lives in a plugin package, proving the platform's separation of concerns for future DSLs.

Non-Goals (for this release)

- No arbitrary JavaScript execution inside graphs.
- No claim that AI output is correct by construction — AI assists authoring, it does not replace validation.
- No support for non-backend domains yet (AI workflows, ETL, rendering, infra) — architecture must not preclude them later.
- No multi-user live collaboration (tracked as Future Work).
- No general-purpose visual scripting language.

---

4. Target Users & Use Cases

Persona	Need	Example Use Case
Backend engineer	Move faster on CRUD-heavy or integration-heavy services without hand-writing boilerplate	Design a REST API with 6 entities, auth middleware, and 3 external webhooks; export a working Express/Fastify service
Solutions/founding engineer	Prototype a backend fast, iterate on shape before committing to code	Sketch an MVP backend, regenerate as requirements shift
Technical PM / architect	Communicate and validate system design without writing code	Build the graph as a living spec; hand off to engineers, or generate a scaffold they refine
AI-assisted builder	Describe a backend in natural language and get a starting point	Prompt: "REST API for a bookstore with orders, inventory, and Stripe checkout" → generated graph → review → compile

---

5. User Journeys

5.1 Manual design → code

1. User opens a new graph, drags in node types (HTTP Route, Entity, Repository, Transform, Response).
2. User wires execution and data edges between nodes.
3. Editor performs live type-checking and contract validation as the user connects ports.
4. User clicks **Compile**. Compiler runs semantic analysis → validation → optimization → code generation.
5. User reviews generated project (routes, models, migrations, tests) and downloads/exports it.

5.2 AI-assisted design → review → code

1. User types a prompt describing the backend they want.
2. AI layer generates a graph (nodes + edges) inside the existing structural graph model.
3. Graph appears in the editor exactly as if hand-built — same nodes, same validation rules apply.
4. User edits, corrects, or extends the AI-generated graph manually.
5. Compilation proceeds identically to the manual path — **AI never bypasses validation or compiles directly**.

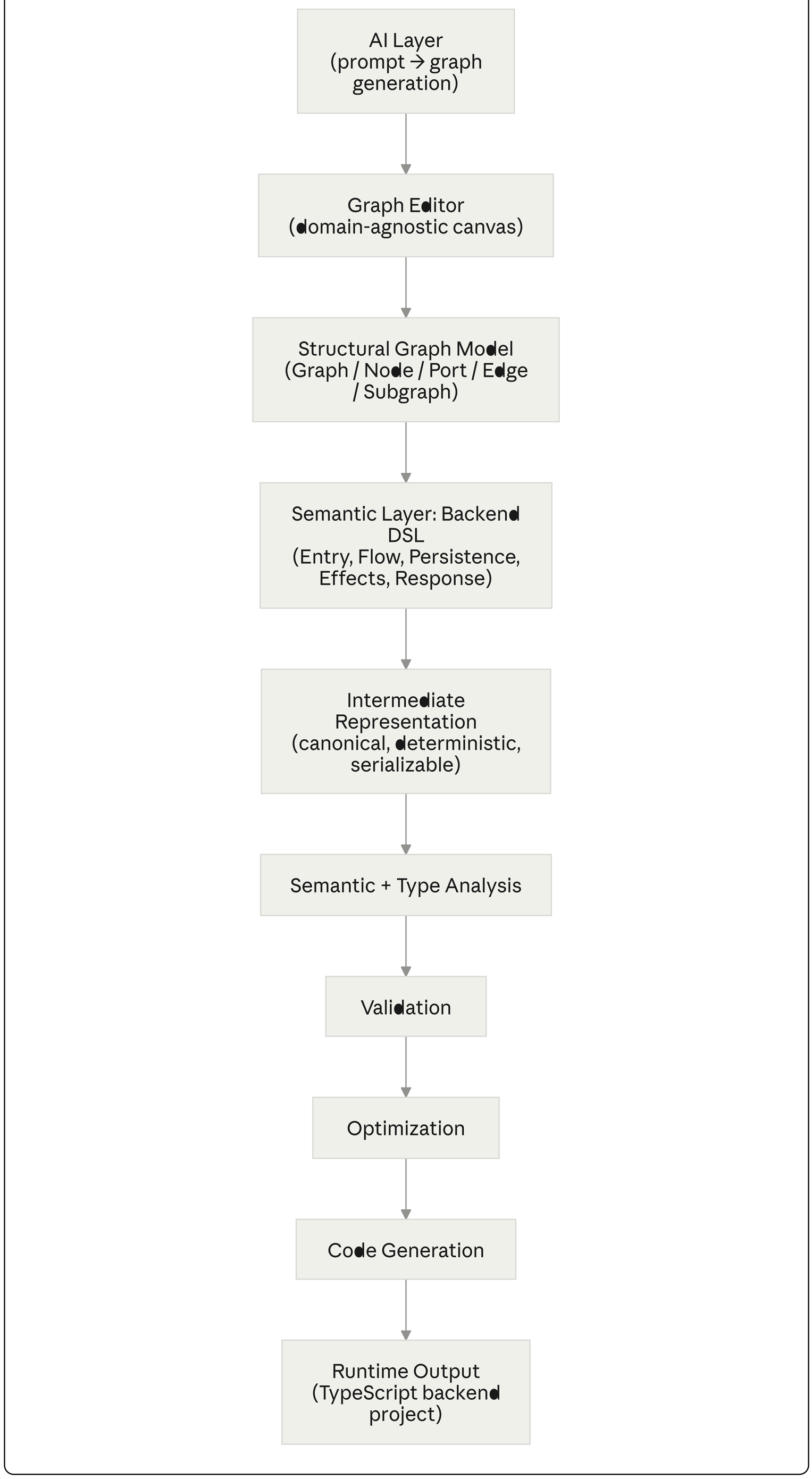
5.3 Iteration

1. User modifies an existing graph (e.g., adds a new field, a new route).
2. Editor re-validates only the affected subgraph (incremental validation).
3. User recompiles; compiler regenerates only the affected output (incremental compilation, stretch goal for v1).

---

6. Product Architecture (adapted from RFC-0001)

The product implements four of the RFC's layers for this release, with Layer 2 scoped to a single package (Backend DSL):



Key invariant carried over from the RFC: the editor never compiles directly to runtime code, and AI never compiles, executes, or defines semantics — it only produces graphs that pass through the same pipeline as manual edits.

---

7. Functional Requirements

7.1 Graph Editor

- FR1: Canvas supports adding, connecting, deleting, and grouping nodes.
- FR2: Ports are typed; invalid connections are rejected or flagged at edit time.
- FR3: Execution edges (control flow) and data edges (values) are visually distinct.
- FR4: Subgraphs support reusable, composable node groups (function-equivalent).
- FR5: Undo/redo, copy/paste, multi-select.
- FR6: Graph serializes to a deterministic, versioned, diff-friendly document (JSON).

7.2 Backend DSL (Layer 2 plugin)

Node families required for v1, scoped from RFC §19:

Family	Node Types (v1 scope)
Entry	HTTP Route, Cron
Flow	Branch, Merge, Transform, Loop, Subgraph
Persistence	Entity, Repository, Query, Mutation
Effects	HTTP Request, Logging, Storage
Response	Success Response, Error Response
Error Handling	Try/Catch, Validation Error

(WebSocket, Queue, GraphQL, RPC, Transactions, and Migrations are Phase 2 additions — see §10.)

- FR7: Each node exposes a contract (inputs, outputs, capabilities, side effects, guarantees) per RFC §10.
- FR8: Capabilities (Queryable, Writable, Pure, Impure, Async, etc.) are resolved during semantic analysis and surfaced to the user (e.g., warn if a "Pure" node has a database write attached).

7.3 Compiler

- FR9: Compiler pipeline runs: semantic analysis → type inference → contract validation → capability resolution → graph validation → optimization → IR generation → code generation.
- FR10: Compilation is deterministic — identical graphs always produce identical output.
- FR11: Validation surfaces errors with node-level references before code generation is attempted (cycles, missing dependencies, type errors, contract violations, dead/unreachable nodes).
- FR12: Code generator targets a TypeScript backend (Express or Fastify — see open question in §12) with generated routes, models, and a migration scaffold.
- FR13: Generated code is human-readable, formatted, and includes comments linking back to source node IDs for traceability.

7.4 AI Layer

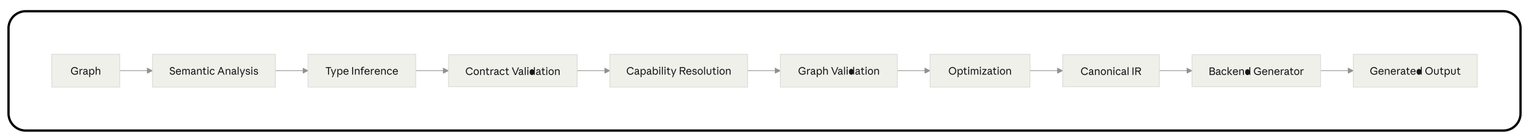
- FR14: User can submit a natural-language prompt; AI returns a graph using only valid node types/contracts from the Backend DSL package.
- FR15: AI-generated graphs are inserted into the same editor state as manual graphs — no special-cased "AI graph" object.
- FR16: AI can also be invoked on an existing graph to: explain a subgraph, suggest a fix for a validation error, or propose (not apply) an optimization.
- FR17: AI output that fails validation is shown with the same error surfaces as manual mistakes — it is never auto-accepted.

7.5 Export / Output

- FR18: User can export the generated project as a downloadable code archive.
- FR19: Generated project includes a README describing how to run it, generated from the graph's routes and entities.

---

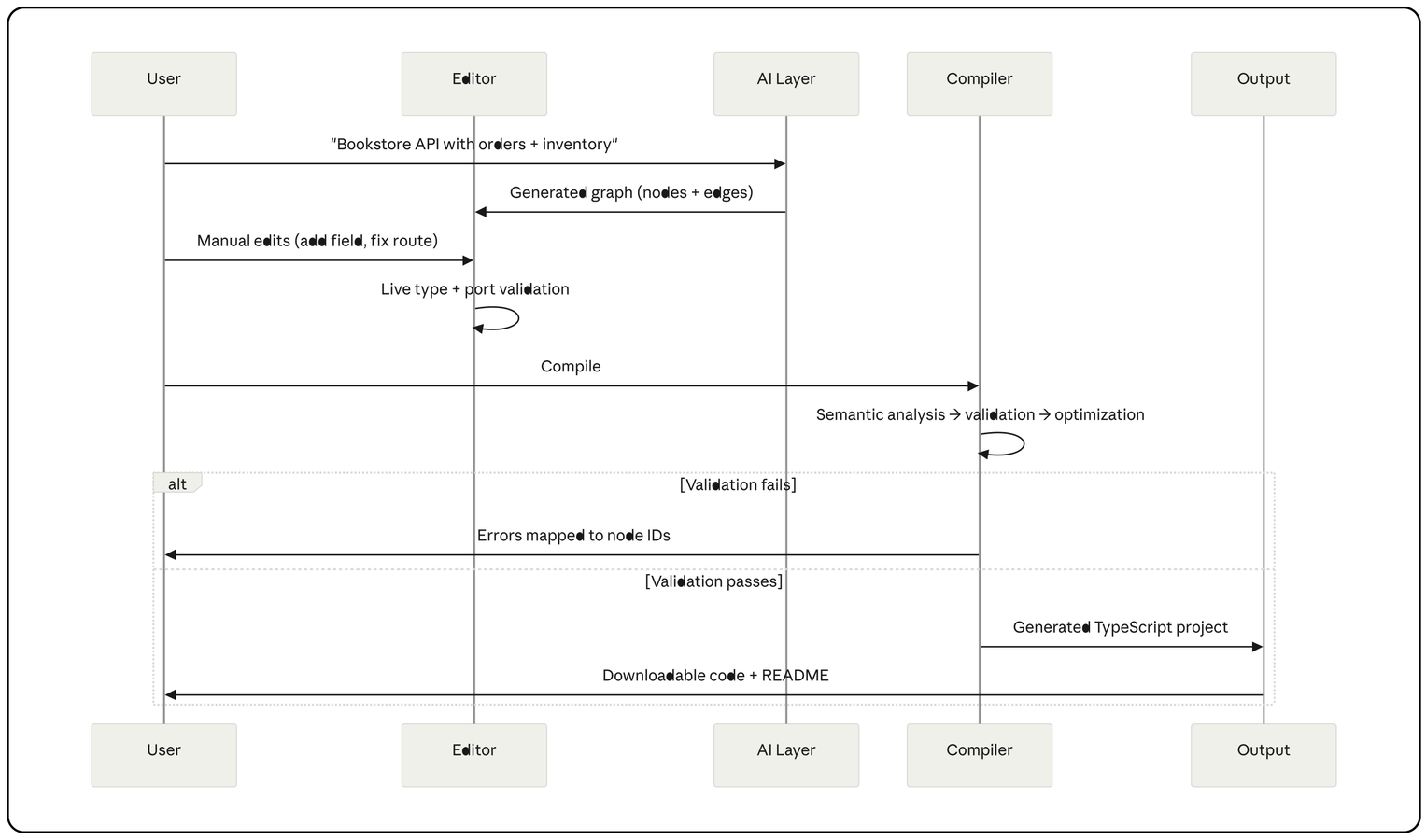
8. Compiler Pipeline (detail)



Validation (RFC §15) must catch, before any code is written: cycles, invalid ports, missing dependencies, type errors, contract violations, capability violations, dead nodes, dead branches, and unreachable graphs.

---

9. Example: Data Flow Through the System



---

10. Release Phasing (mapped from RFC §27 Roadmap)

Phase	Scope	Maps to RFC Phase
P1 — Editor Foundation	Structural graph model, canvas, serialization, undo/redo	RFC Phase 1
P2 — Typing & Validation	Type system, contract validation, execution/data graph separation	RFC Phase 2
P3 — Compiler Core	IR, semantic analysis, optimization passes, diagnostics	RFC Phase 3
P4 — Backend DSL v1 (this PRD's primary deliverable)	Entry/Flow/Persistence/Effects/Response nodes, REST generation, basic validation	RFC Phase 4
P5 — AI Integration v1	Prompt → graph generation, explain/fix suggestions	RFC Phase 6 (pulled forward, scoped down)
P6 — Plugin SDK / Hardening	Public extension points for future DSLs	RFC Phase 5

Transactions, migrations as first-class nodes, WebSocket/Queue entry points, and multi-DSL composition are explicitly deferred past v1 (RFC Phase 4 full scope, Phase 7).

---

11. Out of Scope for v1

- Non-backend DSLs (AI workflow graphs, ETL, rendering, infra, game logic).
- Multiplayer/live collaborative editing.
- Distributed or cloud compilation.
- Formal verification / static analysis beyond compile-time validation.
- Graph package manager / marketplace.
- Visual debugger and execution profiler for generated runtimes.
- Custom/pluggable runtime targets beyond the initial TypeScript target.

---

12. Open Questions

- Which backend runtime target ships first — Express, Fastify, or a framework-agnostic core with adapters?
- Should the AI layer be allowed to run multiple generation passes (draft → self-critique → refine) before returning a graph, or must every AI turn produce a single reviewable graph?
- Pull-based vs. push-based execution model for the generated runtime's data graph (RFC open question, inherited here)?

- How much of the generated code should be regenerated vs. preserved across recompiles once a user hand-edits the output? (Round-tripping is a known hard problem for code-gen tools.)
- Should validation errors block compilation entirely, or allow "compile with warnings" for partial scaffolds?
- What's the minimum contract surface (RFC §10) required per node before we consider the Backend DSL "complete" for v1 — is capability resolution (§11) required at launch or can it ship as a fast-follow?

---

13. Success Metrics

- **Time to first working backend:** median time from empty graph (or AI prompt) to a successfully compiled, runnable project.
- **Validation catch rate:** % of design errors caught at compile time vs. discovered only after running generated code.
- **AI-assisted graph acceptance rate:** % of AI-generated graphs a user compiles with zero or minor manual edits.
- **Generated code retention:** % of users who continue working directly in the generated codebase after export (proxy for code quality/readability).

---

14. Risks

Risk	Mitigation
AI-generated graphs pass validation but produce semantically wrong backends (e.g., wrong side effects)	Keep AI strictly graph-only, never semantics-defining (RFC §20); require human review before compile is treated as "final"
Node/contract surface grows unbounded and the DSL becomes as complex as hand-written code	Cap v1 node families to §7.2 list; require RFC-style contract review before adding new node types
Generated code becomes unmaintainable outside the tool	Enforce readability requirements (FR13) as a hard release gate, tested via golden-output tests
Editor accumulates domain knowledge, violating the domain-agnostic core principle	Architectural review gate: any PR adding backend-specific logic to the editor (vs. the DSL plugin) is rejected